PYTHON ACTIVITY BASED

1.Hospital Billing and Patient Classification

1. Problem Statement

To develop a Python program that classifies patients based on their age and calculates hospital bills according to the selected ward and number of days admitted.

The program classifies patients as children, adults, or senior citizens. It provides three ward options: General Ward, Special Ward, and ICU. Based on the selected ward and duration of stay, the program calculates and displays the total hospital bill.

2. Objectives

- To classify patients based on their age.
- To understand the use of if-elif-else statements in Python.
- To calculate hospital charges based on ward selection.
- To accept patient details and hospital stay information.
- To display the patient's name, total days, and final bill.
- To handle invalid ward choices.

3. Assumptions

1. The patient's name is entered as text.
2. The patient's age is entered as an integer in years.
3. Patients below 18 years are classified as children.
4. Patients aged 18 to 59 years are classified as adults.
5. Patients aged 60 years and above are classified as senior citizens.
6. The hospital charges are fixed as follows:

Ward type	Charge per day
General Ward	₹1,000
Special Ward	₹2,000
ICU	₹5,000

7. The number of hospital days is entered as an integer.
8. No additional charges, taxes, or discounts are included.
9. These charges are sample values for the assignment, not actual hospital rates.

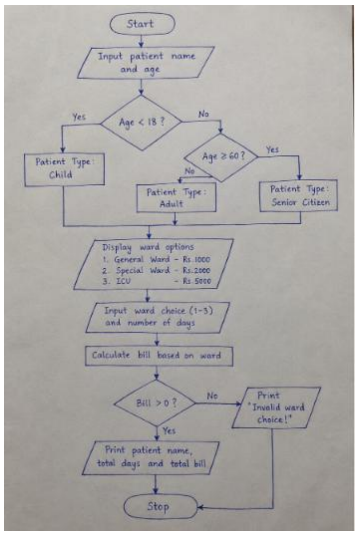
4. Algorithm

1. Start the program.
2. Read the patient's name and age.
3. Check the patient's age:
    - If age is below 18, classify as Child.

- If age is 60 or above, classify as Senior Citizen.
- Otherwise, classify as Adult.

4. Display the three ward options and their charges.
5. Read the ward choice and number of days.
6. Calculate the bill based on the selected ward:
   - General Ward: ₹1,000 × days.
   - Special Ward: ₹2,000 × days.
   - ICU: ₹5,000 × days.
7. If the ward choice is invalid, display an error message.
8. Otherwise, display the patient's name, number of days, and total bill.
9. Stop the program.

5. Flowchart



6. Pseudocode

```
START
Input patient_name
Input age
if age < 18 then
  display "Child"
else if age >= 60 then
  display "Senior Citizen"
else
  display "Adult"
end if
display ward options
input choice
```

```
input days

if choice == 1 then

   bill = 1000 * days

else if choice == 2 then

   bill = 2000 * days

else if choice == 3 then

   bill = 5000 * days

else

   bill = 0

display "Invalid ward choice"

end if

if bill > 0 then

   display patient_name

   display days

display bill

end if

STOP
```

7.Code

```
name = input("Enter patient name: ")
age = int(input("Enter patient age: "))

print("\nPatient Classification")
if age < 18:
   print("Patient Type: Child")
elif age >= 60:
   print("Patient Type: Senior Citizen")
else:
   print("Patient Type: Adult")

print("\nHospital Billing")
print("1. General Ward - Rs. 1000")
print("2. Special Ward - Rs. 2000")
print("3. ICU - Rs. 5000")

choice = int(input("Enter ward choice (1-3): "))
days = int(input("Enter number of days: "))

if choice == 1:
   bill = 1000 * days
elif choice == 2:
   bill = 2000 * days
elif choice == 3:
   bill = 5000 * days
else:
   bill = 0
   print("Invalid ward choice!")
```

```
if bill > 0:
   print("\n--- Hospital Bill ---")
   print("Patient Name:", name)
   print("Total Days:", days)
   print("Total Bill: Rs.", bill)
```

8. Test Cases

Test	Age	Ward choice	Days	Expected result
1	10	General Ward (1)	2	Child; bill ₹2,000
2	30	Special Ward (2)	3	Adult; bill ₹6,000
3	65	ICU (3)	2	Senior Citizen; bill ₹10,000
4	25	Invalid choice (4)	2	Invalid ward choice
5	17	General Ward (1)	5	Child; bill ₹5,000

These expected results follow the supplied program, assuming a positive number of hospital days.

9. Output Screenshot

```
Enter patient name: ravi
Enter patient age: 45

Patient Classification
Patient Type: Adult

Hospital Billing
1. General Ward - Rs. 1000
2. Special Ward - Rs. 2000
3. ICU - Rs. 5000
Enter ward choice (1-3): 2
Enter number of days: 75

--- Hospital Bill ---
Patient Name: ravi
Total Days: 75
Total Bill: Rs. 150000
```

10. Conclusion

The Hospital Billing and Patient Classification program successfully classifies patients according to their age and calculates hospital bills based on ward selection and the number of days admitted. It demonstrates basic Python concepts such as input, output, arithmetic operations, and conditional statements. This program helps students understand how simple billing systems can be developed using Python.

2.Retail Billing with Multiple Discount Rules

1. Problem Statement

To develop a Python program that calculates the final bill amount for a customer based on the total purchase amount and applicable discount rules.

The program accepts the customer's name and purchase amount, determines the discount percentage using conditional statements, calculates the discount amount, and displays the final bill.

2. Objectives

- To calculate the discount based on the purchase amount.
- To understand if-elif-else statements in Python.
- To calculate the discount amount and final bill.
- To accept customer details using input().
- To display the purchase amount, discount, and final bill.

3. Assumptions

1. The customer enters their name and total purchase amount.
2. The purchase amount is entered in Indian Rupees.
3. Discounts are applied according to the following rules:

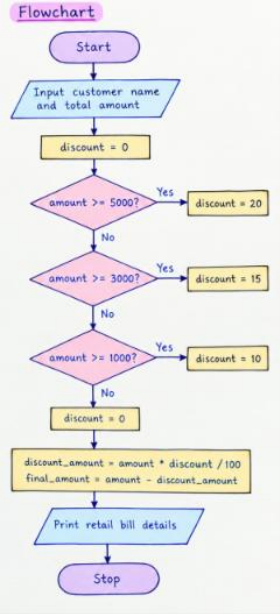
Purchase amount	Discount
₹5,000 and above	20%
₹3,000 to ₹4,999.99	15%
₹1,000 to ₹2,999.99	10%
Below ₹1,000	0%

4. Only one discount rate is applied to the total purchase amount.
5. No additional taxes or charges are included.
6. The program uses sample discount rules for educational purposes.

4. Algorithm

1. Start the program.
2. Read the customer's name.
3. Read the total purchase amount.
4. If the amount is ₹5,000 or above, assign a 20% discount.
5. Otherwise, if the amount is ₹3,000 or above, assign a 15% discount.
6. Otherwise, if the amount is ₹1,000 or above, assign a 10% discount.
7. Otherwise, assign no discount.
8. Calculate the discount amount using the purchase amount and discount percentage.
9. Subtract the discount amount from the purchase amount.
10. Display the customer name, purchase amount, discount percentage, discount amount, and final bill.
11. Stop the program.

5.Flowchart



6. Pseudocode

START

INPUT customer_name

INPUT amount

IF amount >= 5000 THEN

  discount = 20

ELSE IF amount >= 3000 THEN

  discount = 15

ELSE IF amount >= 1000 THEN

  discount = 10

ELSE

  discount = 0

END IF

discount_amount = amount * discount / 100

final_amount = amount - discount_amount

DISPLAY customer_name

DISPLAY amount

DISPLAY discount

DISPLAY discount_amount

DISPLAY final_amount

STOP

7.Code

```
name = input("Enter customer name: ")
amount = float(input("Enter total purchase amount: "))
discount = 0

if amount >= 5000:
  discount = 20
elif amount >= 3000:
  discount = 15
elif amount >= 1000:
  discount = 10
else:
  discount = 0

discount_amount = amount * discount / 100
final_amount = amount - discount_amount

print("\n----- RETAIL BILL -----")
print("Customer Name:", name)
print("Purchase Amount: Rs.", amount)
print("Discount:", discount, "%")
print("Discount Amount: Rs.", discount_amount)
print("Final Bill Amount: Rs.", final_amount)
```

8. Test Cases

Test	Purchase amount	Discount	Discount amount	Final bill
1	₹6,000	20%	₹1,200	₹4,800
2	₹4,000	15%	₹600	₹3,400
3	₹2,000	10%	₹200	₹1,800
4	₹800	0%	₹0	₹800
5	₹5,000	20%	₹1,000	₹4,000

These test cases verify that the program applies the correct discount at each threshold.

9. Output Screenshot

```
Enter customer name: mahitha
Enter total purchase amount: 4000

----- RETAIL BILL -----
Customer Name: mahitha
Purchase Amount: Rs. 4000.0
Discount: 15 %
Discount Amount: Rs. 600.0
Final Bill Amount: Rs. 3400.0
```

10. Conclusion

The Retail Billing with Multiple Discount Rules program calculates discounts based on the customer's total purchase amount. It uses conditional statements to select the discount percentage and arithmetic operations to calculate the discount amount and final bill. This program helps students understand decision-making and basic calculations in Python.

3.Multi-Currency Travel Budget Analysis

1. Problem Statement

To develop a Python program that converts a travel budget from Indian Rupees (INR) into a selected foreign currency, such as US Dollars (USD), Euros (EUR), or British Pounds (GBP). The program uses fixed exchange rates and displays the converted budget.

2. Objectives

- To understand the use of conditional statements in Python.
- To accept input from the user.
- To use if, elif, and else statements.
- To perform currency conversion using arithmetic operations.
- To display the converted budget in the selected currency.

3. Assumptions

1. The user enters the travel budget in Indian Rupees (INR).
2. The budget is entered as a positive number.
3. The user selects one of the three available currencies.
4. The exchange rates are fixed sample values.
5. The program displays an error message if the user enters an invalid currency choice.

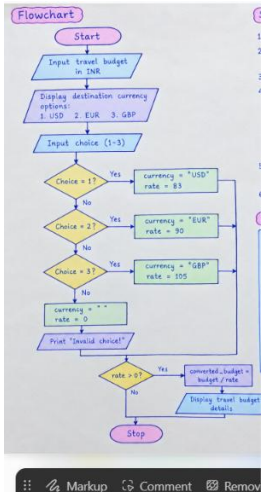
Currency	Exchange rate
US Dollar (USD)	1 USD = ₹83
Euro (EUR)	1 EUR = ₹90
British Pound (GBP)	1 GBP = ₹105

4. Algorithm

1. Start the program.

2. Display the program title.
3. Read the travel budget in INR.
4. Display the three currency options.
5. Read the user's choice.
6. If the choice is 1, select USD and set the rate to 83.
7. If the choice is 2, select EUR and set the rate to 90.
8. If the choice is 3, select GBP and set the rate to 105.
9. Otherwise, display “Invalid choice!”.
10. If the exchange rate is greater than zero, calculate the converted budget.
11. Display the budget, selected currency, exchange rate, and converted amount.
12. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  DISPLAY program title
  INPUT budget in INR

  DISPLAY currency options
  INPUT choice

  IF choice = 1 THEN
    currency = "USD"
    rate = 83
  ELSE IF choice = 2 THEN
    currency = "EUR"
```

```
    rate = 90
  ELSE IF choice = 3 THEN
    currency = "GBP"
    rate = 105
  ELSE
    rate = 0
    DISPLAY "Invalid choice!"
  END IF

  IF rate > 0 THEN
    converted_budget = budget / rate
    DISPLAY budget
    DISPLAY currency
    DISPLAY rate
    DISPLAY converted_budget
  END IF
END
```

7.Code

```
print("Multi-Currency Travel Budget Analysis")
budget = float(input("Enter your travel budget in INR: "))
print("\nSelect destination currency:")
print("1. US Dollar (USD)")
print("2. Euro (EUR)")
print("3. British Pound (GBP)")
choice = int(input("Enter your choice (1-3): "))
if choice == 1:
  currency = "USD"
  rate = 83
elif choice == 2:
  currency = "EUR"
  rate = 90
elif choice == 3:
  currency = "GBP"
  rate = 105
else:
```

```
    currency = ""
    rate = 0
    print("Invalid choice!")

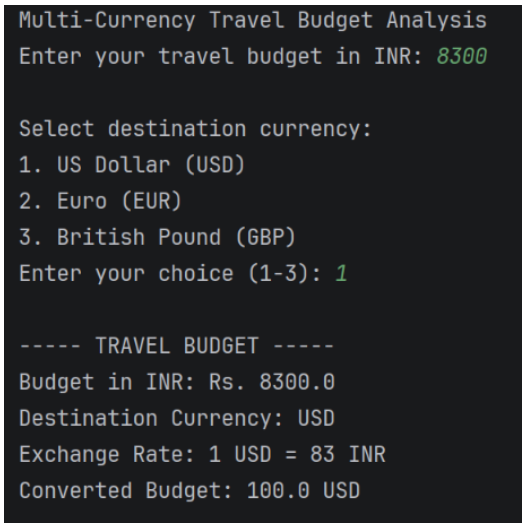
if rate > 0:
    converted_budget = budget / rate

    print("\n----- TRAVEL BUDGET -----")
    print("Budget in INR: Rs.", budget)
    print("Destination Currency:", currency)
    print("Exchange Rate: 1", currency, "=", rate, "INR")
    print("Converted Budget:", round(converted_budget, 2), currency)
```

8. Test Cases

Test	Budget (INR)	Choice	Expected result
1	8300	1	100.00 USD
2	9000	2	100.00 EUR
3	10500	3	100.00 GBP
4	5000	4	Invalid choice!

9. Output Screenshot



10. Conclusion

The Multi-Currency Travel Budget Analysis program successfully converts an Indian Rupee budget into USD, EUR, or GBP using fixed exchange rates. It demonstrates user input, conditional statements, arithmetic calculations, and output formatting in Python. It also handles invalid currency selections.

4.Examination Result and Grace-Mark Validation

1. Problem Statement

To develop a Python program that checks a student's examination marks out of 100 and displays whether the student has passed, passed with grace marks, failed, or entered invalid marks.

2. Objectives

- To validate marks entered by the user.
- To use if, elif, and else statements.
- To calculate the grace marks required.
- To display the examination result.

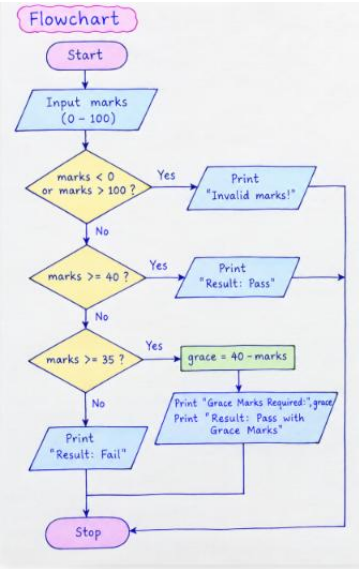
3. Assumptions

1. The student enters marks as an integer.
2. The maximum marks are 100.
3. The minimum passing marks are 40.
4. Students scoring 35–39 are eligible for grace marks.
5. Marks below 0 or above 100 are invalid.

4. Algorithm

1. Start.
2. Display the program title.
3. Input marks out of 100.
4. If marks are below 0 or above 100, display “Invalid marks!”.
5. Otherwise, if marks are 40 or above, display “Pass”.
6. Otherwise, if marks are 35 or above, calculate grace marks using 40 - marks.
7. Display the grace marks required and “Pass with Grace Marks”.
8. Otherwise, display “Fail”.
9. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  INPUT marks

  IF marks < 0 OR marks > 100 THEN
    DISPLAY "Invalid marks!"
  ELSE IF marks >= 40 THEN
    DISPLAY "Result: Pass"
  ELSE IF marks >= 35 THEN
    grace = 40 - marks
    DISPLAY grace
    DISPLAY "Pass with Grace Marks"
  ELSE
    DISPLAY "Result: Fail"
  END IF
END
```

7.Code

```
print("Examination Result and Grace-Mark Validation")
marks = int(input("Enter your marks out of 100: "))
if marks < 0 or marks > 100:
  print("Invalid marks!")
elif marks >= 40:
  print("Result: Pass")
elif marks >= 35:
```

```
    grace = 40 - marks
    print("Grace Marks Required:", grace)
    print("Result: Pass with Grace Marks")
else:
    print("Result: Fail")
```

8. Test Cases

Marks	Expected output
-5	Invalid marks!
105	Invalid marks!
75	Result: Pass
38	Grace Marks Required: 2; Pass with Grace Marks
35	Grace Marks Required: 5; Pass with Grace Marks
25	Result: Fail

9. Sample Output

```
Examination Result and Grace-Mark Validation
Enter your marks out of 100: 38
Grace Marks Required: 2
Result: Pass with Grace Marks
```

10. Conclusion

The program checks whether the entered marks are valid and determines the student's result. It calculates the grace marks required for students scoring between 35 and 39 and displays the appropriate result using Python conditional statements.

5.ATM Transaction Simulation – Python Assignment

1. Problem Statement

To develop a Python program that simulates basic ATM transactions such as checking the account balance, depositing money, and withdrawing money. The program verifies the user's PIN before allowing transactions.

2. Objectives

- To understand conditional statements in Python.
- To verify the ATM PIN.
- To check the available balance.
- To deposit and withdraw money.
- To validate transaction amounts and display the updated balance.

3. Assumptions

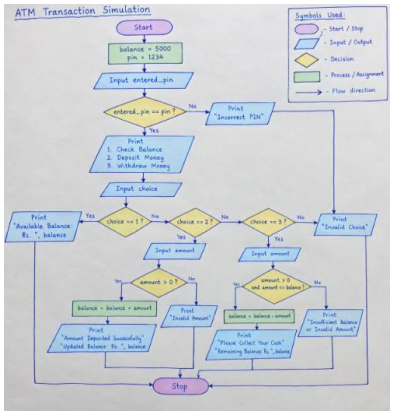
1. The initial account balance is ₹5,000.

2. The ATM PIN is 1234.
3. The user can select one of three operations.
4. Deposit and withdrawal amounts must be greater than zero.
5. Withdrawal is allowed only when sufficient balance is available.
6. This is a basic educational simulation, not a real banking system.

4. Algorithm

1. Start.
2. Set balance to ₹5,000 and PIN to 1234.
3. Ask the user to enter the ATM PIN.
4. If the PIN is incorrect, display “Incorrect PIN”.
5. If the PIN is correct, display the ATM menu.
6. Ask the user to select an operation.
7. If choice is 1, display the available balance.
8. If choice is 2, input the deposit amount.
9. If the deposit amount is greater than zero, add it to the balance and display the updated balance.
10. If choice is 3, input the withdrawal amount.
11. If the amount is positive and does not exceed the balance, subtract it and display the remaining balance.
12. Otherwise, display an appropriate error message.
13. If the menu choice is invalid, display “Invalid Choice”.
14. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  balance = 5000
  pin = 1234

  INPUT entered_pin
```

```
  IF entered_pin = pin THEN
    DISPLAY ATM menu
    INPUT choice

    IF choice = 1 THEN
      DISPLAY balance

    ELSE IF choice = 2 THEN
      INPUT amount
      IF amount > 0 THEN
        balance = balance + amount
        DISPLAY updated balance
      ELSE
        DISPLAY "Invalid Amount"
      END IF

    ELSE IF choice = 3 THEN
      INPUT amount
      IF amount > 0 AND amount <= balance THEN
        balance = balance - amount
        DISPLAY remaining balance
      ELSE
        DISPLAY "Insufficient Balance or Invalid Amount"
      END IF

    ELSE
      DISPLAY "Invalid Choice"
    END IF

  ELSE
    DISPLAY "Incorrect PIN"
  END IF
END
```

7.Code

```
print("ATM Transaction Simulation")
balance = 5000
pin = 1234
entered_pin = int(input("Enter your ATM PIN: "))
if entered_pin == pin:
  print("\n1. Check Balance")
  print("2. Deposit Money")
  print("3. Withdraw Money")

 choice = int(input("Enter your choice: "))

  if choice == 1:
    print("Available Balance: Rs.", balance)

 elif choice == 2:
    amount = int(input("Enter deposit amount: "))
    if amount > 0:
      balance = balance + amount
      print("Amount Deposited Successfully")
      print("Updated Balance: Rs.", balance)
    else:
      print("Invalid Amount")

  elif choice == 3:
    amount = int(input("Enter withdrawal amount: "))
    if amount > 0 and amount <= balance:
      balance = balance - amount
      print("Please Collect Your Cash")
      print("Remaining Balance: Rs.", balance)
    else:
      print("Insufficient Balance or Invalid Amount")

  else:
    print("Invalid Choice")
```

else:

print("Incorrect PIN")

8. Test Cases

Test	Input	Expected output
1	PIN: 1234, choice: 1	Available Balance: ₹5,000
2	PIN: 1234, choice: 2, amount: 1000	Updated Balance: ₹6,000
3	PIN: 1234, choice: 3, amount: 2000	Remaining Balance: ₹3,000
4	PIN: 1111	Incorrect PIN
5	PIN: 1234, choice: 2, amount: -100	Invalid Amount
6	PIN: 1234, choice: 3, amount: 6000	Insufficient Balance or Invalid Amount
7	PIN: 1234, choice: 5	Invalid Choice

Each test runs independently with the initial balance of ₹5,000.

9. Sample Output

```
ATM Transaction Simulation
Enter your ATM PIN: 1234

1. Check Balance
2. Deposit Money
3. Withdraw Money
Enter your choice: 2000
Invalid Choice
```

10. Conclusion

The ATM Transaction Simulation program demonstrates how Python conditional statements can be used to verify a PIN and perform basic banking operations. It checks transaction amounts, prevents withdrawals exceeding the available balance, and displays the appropriate messages for valid and invalid operations.

6. Progressive Water Tariff Analysis – Python Assignment

1. Problem Statement

To develop a Python program that calculates a water bill based on the number of units consumed. The program applies different rates to different consumption slabs and displays the total bill amount.

2. Objectives

- To calculate water charges based on consumption.
- To use if, elif, and else statements.
- To apply progressive tariff rates.
- To validate water consumption input.
- To display the final water bill.

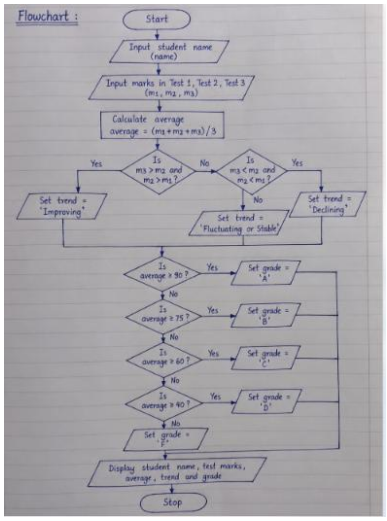
3. Assumptions

1. Water consumption is entered in units.
2. Negative consumption is invalid.
3. The first 100 units cost ₹2 per unit.
4. Units from 101 to 200 cost ₹3 per unit.
5. Units from 201 to 300 cost ₹5 per unit.
6. Units above 300 cost ₹8 per unit.
7. Each rate applies only to the units in its slab.

4. Algorithm

1. Start.
2. Display the program title.
3. Input water consumption units.
4. If units are less than zero, display “Invalid water consumption”.
5. If units are at most 100, calculate the bill at ₹2 per unit.
6. If units are at most 200, charge ₹2 for the first 100 units and ₹3 for the remaining units.
7. If units are at most 300, charge the first 100 units at ₹2, the next 100 at ₹3, and the remaining units at ₹5.
8. If units exceed 300, calculate charges for all four slabs.
9. Display consumption and the total bill.
10. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  INPUT units

  IF units < 0 THEN
```

```
    DISPLAY "Invalid water consumption"

  ELSE IF units <= 100 THEN
    bill = units * 2

  ELSE IF units <= 200 THEN
    bill = 100 * 2 + (units - 100) * 3

  ELSE IF units <= 300 THEN
    bill = 100 * 2 + 100 * 3
        + (units - 200) * 5

  ELSE
    bill = 100 * 2 + 100 * 3 + 100 * 5
        + (units - 300) * 8
  END IF

  IF units >= 0 THEN
    DISPLAY units
    DISPLAY bill
  END IF
END
```

7.Code

```
print("Progressive Water Tariff Analysis")

units = float(input("Enter water consumption units: "))

if units < 0:
  print("Invalid water consumption")
else:
  if units <= 100:
    bill = units * 2
  elif units <= 200:
    bill = (100 * 2) + (units - 100) * 3
```

```
elif units <= 300:
    bill = (100 * 2) + (100 * 3) + (units - 200) * 5
else:
    bill = (100 * 2) + (100 * 3) + (100 * 5) + (units - 300) * 8


print("\n----- WATER BILL -----")
print("Water Consumption:", units, "units")
print("Total Water Bill: Rs.", round(bill, 2))
```

8. Test Cases

Test	Units	Expected output
1	-10	Invalid water consumption
2	50	₹100
3	150	₹350
4	250	₹750
5	350	₹1,400
6	100	₹200

9. Sample Output

```
Progressive Water Tariff Analysis
Enter water consumption units: 250

----- WATER BILL -----
Water Consumption: 250.0 units
Total Water Bill: Rs. 750.0
```

10. Conclusion

The Progressive Water Tariff Analysis program calculates water charges based on consumption slabs. It demonstrates conditional statements, arithmetic operations, input validation, and output formatting in Python. The progressive billing method ensures that each portion of water consumption is charged at its corresponding rate.

7.Password Policy Validation Engine – Python Assignment

1. Problem Statement

To develop a Python program that checks whether a password follows the required security rules. The password must contain at least 8 characters, one uppercase letter, one lowercase letter, one number, and one special character.

2. Objectives

- To validate password strength.
- To use conditional statements in Python.
- To check uppercase and lowercase letters.

- To check digits and special characters.
- To display appropriate validation messages.

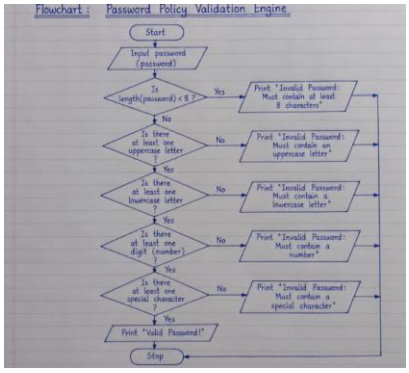
3. Assumptions

1. The password is entered by the user.
2. The minimum password length is 8 characters.
3. At least one uppercase letter is required.
4. At least one lowercase letter is required.
5. At least one digit is required.
6. At least one special character is required, such as @, #, $, or !.
7. The program displays the first validation error it finds.

4. Algorithm

1. Start.
2. Display the program title.
3. Input the password.
4. Check whether the password length is less than 8.
5. If yes, display the minimum-length error.
6. Otherwise, check for an uppercase letter.
7. If missing, display the uppercase-letter error.
8. Otherwise, check for a lowercase letter.
9. If missing, display the lowercase-letter error.
10. Otherwise, check for a digit.
11. If missing, display the number error.
12. Otherwise, check for a special character.
13. If missing, display the special-character error.
14. If all conditions are satisfied, display “Valid Password!”.
15. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  INPUT password

  IF length(password) < 8 THEN
    DISPLAY "Must contain at least 8 characters"

  ELSE IF no uppercase letter THEN
    DISPLAY "Must contain an uppercase letter"

  ELSE IF no lowercase letter THEN
    DISPLAY "Must contain a lowercase letter"

  ELSE IF no digit THEN
    DISPLAY "Must contain a number"

  ELSE IF no special character THEN
    DISPLAY "Must contain a special character"

  ELSE
    DISPLAY "Valid Password!"
  END IF
END
```

7.Code

```
print("Password Policy Validation Engine")
password = input("Enter your password: ")
if len(password) < 8:
  print("Invalid Password: Must contain at least 8 characters")
elif not any(ch.isupper() for ch in password):
  print("Invalid Password: Must contain an uppercase letter")
elif not any(ch.islower() for ch in password):
  print("Invalid Password: Must contain a lowercase letter")
elif not any(ch.isdigit() for ch in password):
  print("Invalid Password: Must contain a number")
elif not any(not ch.isalnum() for ch in password):
  print("Invalid Password: Must contain a special character")
```

```
else:
    print("Valid Password!")
```

8. Test Cases

Test password	Expected output
Ab1@	Invalid: Must contain at least 8 characters
abcdefgh1@	Invalid: Must contain an uppercase letter
ABCDEFGH1@	Invalid: Must contain a lowercase letter
Abcdefgh@	Invalid: Must contain a number
Abcdefg1	Invalid: Must contain a special character
Abcdef1@	Invalid: Must contain at least 8 characters
Abcdefg1@	Valid Password!

9. Sample Output

```
Password Policy Validation Engine
Enter your password: mguyd
Invalid Password: Must contain at least 8 characters
```

10. Conclusion

The Password Policy Validation Engine checks whether a password satisfies the specified security requirements. It uses Python conditional statements, string functions, and character checks to identify missing requirements and display the appropriate message.

8.Railway Fare and Reservation Calculator

1. Problem Statement

To develop a Python program that calculates railway ticket fares based on passenger age, travel class, and distance. The program applies age-based concessions, adds a reservation charge, and displays the total ticket fare.

2. Objectives

- To calculate railway fares based on travel distance and class.
- To use if, elif, and else statements.
- To apply age-based concessions.
- To add reservation charges.
- To validate class selection and travel distance.

3. Assumptions

1. The passenger enters their name and age.
2. Sleeper class costs ₹1 per km.
3. AC 3 Tier costs ₹2 per km.

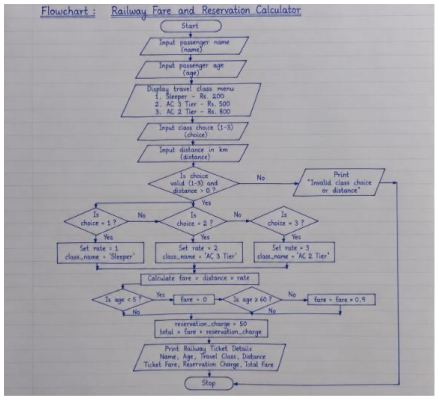
4. AC 2 Tier costs ₹3 per km.
5. Children below 5 years receive a full fare waiver.
6. Senior citizens aged 60 and above receive a 10% fare discount.
7. The reservation charge is ₹50.
8. The distance must be greater than zero.
9. The program uses the fare rules given in the code for this assignment.

Note: The class prices printed in the menu (₹200, ₹500, ₹800) do not match the per-kilometre rates used in the calculation. This assignment follows the rates actually used in the calculation.

4. Algorithm

1. Start.
2. Input passenger name and age.
3. Display the available travel classes.
4. Input class choice and travel distance.
5. If the choice is 1, select Sleeper and set the rate to ₹1 per km.
6. If the choice is 2, select AC 3 Tier and set the rate to ₹2 per km.
7. If the choice is 3, select AC 2 Tier and set the rate to ₹3 per km.
8. If the class choice is invalid or distance is not positive, display an error message.
9. Otherwise, calculate the fare using the selected class rate and distance.
10. If age is below 5, set the fare to zero.
11. If age is 60 or above, apply a 10% discount to the fare.
12. Add the reservation charge of ₹50.
13. Display the passenger details, class, distance, fare, reservation charge, and total fare.
14. Stop.

5. Flowchart



6. Pseudocode

BEGIN

INPUT name

INPUT age

```
DISPLAY travel classes
INPUT choice
INPUT distance

IF choice = 1 THEN
  rate = 1
  class_name = "Sleeper"
ELSE IF choice = 2 THEN
  rate = 2
  class_name = "AC 3 Tier"
ELSE IF choice = 3 THEN
  rate = 3
  class_name = "AC 2 Tier"
ELSE
  rate = 0
  class_name = "Invalid"
END IF

IF rate > 0 AND distance > 0 THEN
  fare = distance * rate

  IF age < 5 THEN
    fare = 0
  ELSE IF age >= 60 THEN
    fare = fare * 0.9
  END IF

  reservation_charge = 50
  total = fare + reservation_charge

  DISPLAY passenger details
  DISPLAY fare
  DISPLAY reservation charge
  DISPLAY total
ELSE
```

```
    DISPLAY "Invalid class choice or distance"
  END IF
END
```

7.Code

```
print("Railway Fare and Reservation Calculator")

name = input("Enter passenger name: ")
age = int(input("Enter passenger age: "))

print("\nSelect Travel Class")
print("1. Sleeper - Rs. 200")
print("2. AC 3 Tier - Rs. 500")
print("3. AC 2 Tier - Rs. 800")

choice = int(input("Enter class choice (1-3): "))
distance = float(input("Enter distance in km: "))

if choice == 1:
  rate = 1
  class_name = "Sleeper"
elif choice == 2:
  rate = 2
  class_name = "AC 3 Tier"
elif choice == 3:
  rate = 3
  class_name = "AC 2 Tier"
else:
  rate = 0
  class_name = "Invalid"

if rate > 0 and distance > 0:
  if choice == 1:
    fare = distance * 1
```

```
    elif choice == 2:
        fare = distance * 2
    else:
        fare = distance * 3

    if age < 5:
        fare = 0
    elif age >= 60:
        fare = fare * 0.9

    reservation_charge = 50
    total = fare + reservation_charge

    print("\n----- RAILWAY TICKET -----")
    print("Passenger Name:", name)
    print("Passenger Age:", age)
    print("Travel Class:", class_name)
    print("Distance:", distance, "km")
    print("Ticket Fare: Rs.", round(fare, 2))
    print("Reservation Charge: Rs.", reservation_charge)
    print("Total Fare: Rs.", round(total, 2))
else:
    print("Invalid class choice or distance")
```

8. Test Cases

Test	Age	Class	Distance	Expected total
1	25	Sleeper	100 km	₹150
2	30	AC 3 Tier	100 km	₹250
3	65	Sleeper	100 km	₹140
4	4	AC 2 Tier	100 km	₹50
5	25	Invalid choice	100 km	Invalid class choice or distance
6	25	Sleeper	0 km	Invalid class choice or distance

9. Sample Output

```
Railway Fare and Reservation Calculator
Enter passenger name: ravi
Enter passenger age: 25

Select Travel Class
1. Sleeper - Rs. 200
2. AC 3 Tier - Rs. 500
3. AC 2 Tier - Rs. 800
Enter class choice (1-3): 2
Enter distance in km: 100

----- RAILWAY TICKET -----
Passenger Name: ravi
Passenger Age: 25
Travel Class: AC 3 Tier
Distance: 100.0 km
Ticket Fare: Rs. 200.0
Reservation Charge: Rs. 50
Total Fare: Rs. 250.0
```

10. Conclusion

The Railway Fare and Reservation Calculator calculates ticket fares based on travel class, distance, and passenger age. It demonstrates conditional statements, arithmetic operations, validation, and fare concessions in Python.

9.Smart Parking Fee Management – Python Assignment

1. Problem Statement

To develop a Python program that calculates parking fees based on the type of vehicle (car or bike) and the number of parking hours. Different rates are applied for the first two hours and the additional hours.

2. Objectives

- To calculate parking fees for cars and bikes.
- To use nested if and elif statements.
- To validate parking hours.
- To apply different rates for different vehicles.
- To display the vehicle type and total parking fee.

3. Assumptions

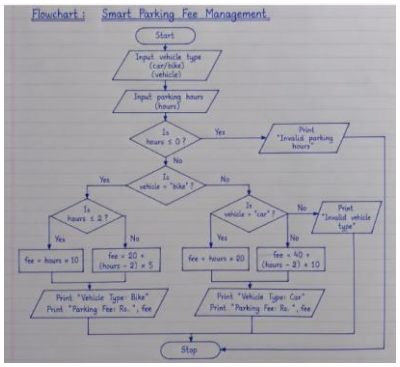
1. The vehicle type must be a car or a bike.
2. Parking hours must be greater than zero.
3. Bike parking costs ₹10 per hour for the first two hours.
4. Additional bike parking costs ₹5 per hour.
5. Car parking costs ₹20 per hour for the first two hours.
6. Additional car parking costs ₹10 per hour.
7. The program accepts whole numbers for parking hours.

4. Algorithm

1. Start.

2. Display the program title.
3. Input vehicle type.
4. Input parking hours.
5. If parking hours are less than or equal to zero, display “Invalid parking hours”.
6. Otherwise, check the vehicle type.
7. If the vehicle is a bike:
    - If hours are at most 2, calculate fee as hours × 10.
    - Otherwise, calculate fee as ₹20 + (hours − 2) × ₹5.
8. If the vehicle is a car:
    - If hours are at most 2, calculate fee as hours × 20.
    - Otherwise, calculate fee as ₹40 + (hours − 2) × ₹10.
9. Display the vehicle type and parking fee.
10. If the vehicle type is invalid, display “Invalid vehicle type”.
11. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  INPUT vehicle
  INPUT hours

  IF hours <= 0 THEN
    DISPLAY "Invalid parking hours"

  ELSE IF vehicle = "bike" THEN
    IF hours <= 2 THEN
      fee = hours * 10
    ELSE
      fee = 20 + (hours - 2) * 5
```

```
    END IF
    DISPLAY vehicle and fee

  ELSE IF vehicle = "car" THEN
    IF hours <= 2 THEN
      fee = hours * 20
    ELSE
      fee = 40 + (hours - 2) * 10
    END IF
    DISPLAY vehicle and fee

  ELSE
    DISPLAY "Invalid vehicle type"
  END IF
END
```

7.Code

```
print("Smart Parking Fee Management")

vehicle = input("Enter vehicle type (car/bike): ").lower()
hours = int(input("Enter parking hours: "))

if hours <= 0:
  print("Invalid parking hours")

elif vehicle == "bike":
  if hours <= 2:
    fee = hours * 10
  else:
    fee = 20 + (hours - 2) * 5

  print("Vehicle Type: Bike")
  print("Parking Fee: Rs.", fee)

elif vehicle == "car":
```

```
    if hours <= 2:
        fee = hours * 20
    else:
        fee = 40 + (hours - 2) * 10

    print("Vehicle Type: Car")
    print("Parking Fee: Rs.", fee)

else:
    print("Invalid vehicle type")
```

8. Test Cases

Test	Vehicle	Hours	Expected output
1	bike	2	Parking Fee: ₹20
2	bike	4	Parking Fee: ₹30
3	car	2	Parking Fee: ₹40
4	car	5	Parking Fee: ₹70
5	car	0	Invalid parking hours
6	bus	3	Invalid vehicle type

9. Sample Output

```
Smart Parking Fee Management
Enter vehicle type (car/bike): car
Enter parking hours: 5
Vehicle Type: Car
Parking Fee: Rs. 70
```

10. Conclusion

The Smart Parking Fee Management program calculates parking charges based on vehicle type and parking duration. It uses nested conditional statements, arithmetic operations, and input validation to calculate and display the correct parking fee.

10.Student Performance Trend Analysis

1. Problem Statement

Develop a Python program to analyze a student's performance in three tests. The program calculates the average marks, identifies whether the student's performance is improving, declining, or fluctuating/stable, and assigns a grade based on the average marks.

2. Objectives

- To accept the student's name and marks in three tests.

- To calculate the average marks.
- To identify the student's performance trend.
- To assign a grade based on the average marks.
- To display a performance report.

3. Assumptions

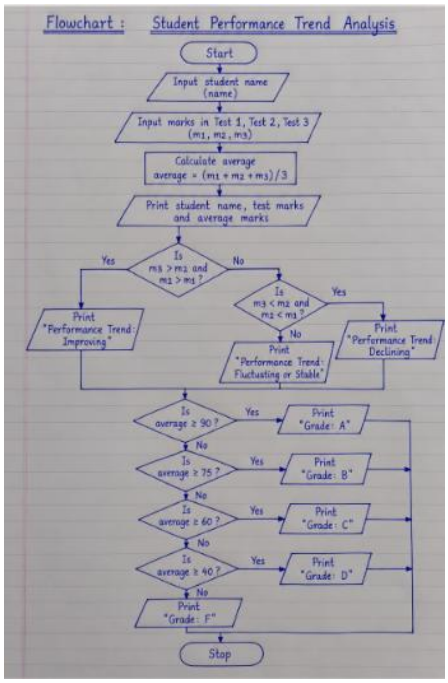
- The student writes three tests.
- Marks are entered as numerical values.
- Marks are assumed to be out of 100.
- The program does not validate negative marks or marks above 100.
- Grades are assigned according to the following criteria.

Average marks	**Grade**
90 and above	A
75–89.99	B
60–74.99	C
40–59.99	D
Below 40	F

4. Algorithm

1. Start the program.
2. Display the title.
3. Read the student's name.
4. Read marks in Test 1, Test 2, and Test 3.
5. Calculate the average of the three tests.
6. Display the student's performance report.
7. If Test 3 marks are greater than Test 2 and Test 2 marks are greater than Test 1, display Improving.
8. Otherwise, if Test 3 marks are less than Test 2 and Test 2 marks are less than Test 1, display Declining.
9. Otherwise, display Fluctuating or Stable.
10. Assign a grade based on the average marks.
11. Display the grade.
12. Stop the program.

5. Flowchart



6. Pseudocode

START

INPUT student name

INPUT m1, m2, m3

average = (m1 + m2 + m3) / 3

DISPLAY student name and test marks

DISPLAY average

IF m3 > m2 AND m2 > m1

  DISPLAY "Improving"

ELSE IF m3 < m2 AND m2 < m1

  DISPLAY "Declining"

ELSE

  DISPLAY "Fluctuating or Stable"

END IF

IF average >= 90

  grade = "A"

ELSE IF average >= 75

```
  grade = "B"
ELSE IF average >= 60
  grade = "C"
ELSE IF average >= 40
  grade = "D"
ELSE
  grade = "F"
END IF

DISPLAY grade

STOP
```

7.Code

```
print("Student Performance Trend Analysis")

name = input("Enter student name: ")

m1 = float(input("Enter marks in Test 1: "))
m2 = float(input("Enter marks in Test 2: "))
m3 = float(input("Enter marks in Test 3: "))

average = (m1 + m2 + m3) / 3

print("\n----- PERFORMANCE REPORT -----")
print("Student Name:", name)
print("Test 1 Marks:", m1)
print("Test 2 Marks:", m2)
print("Test 3 Marks:", m3)
print("Average Marks:", round(average, 2))

if m3 > m2 and m2 > m1:
  print("Performance Trend: Improving")
elif m3 < m2 and m2 < m1:
  print("Performance Trend: Declining")
```

```
else:
    print("Performance Trend: Fluctuating or Stable")


if average >= 90:
    print("Grade: A")
elif average >= 75:
    print("Grade: B")
elif average >= 60:
    print("Grade: C")
elif average >= 40:
    print("Grade: D")
else:
    print("Grade: F")
```

8. Test Cases

Student	Test 1	Test 2	Test 3	Average	Trend	Grade
Ravi	60	70	80	70.00	Improving	C
Sita	90	80	70	80.00	Declining	B
Arun	70	85	75	76.67	Fluctuating or Stable	B
Priya	95	95	95	95.00	Fluctuating or Stable	A
Kiran	30	35	38	34.33	Improving	F

9. Sample Output

```
Student Performance Trend Analysis
Enter student name: ravi
Enter marks in Test 1: 60
Enter marks in Test 2: 70
Enter marks in Test 3: 80

----- PERFORMANCE REPORT -----
Student Name: ravi
Test 1 Marks: 60.0
Test 2 Marks: 70.0
Test 3 Marks: 80.0
Average Marks: 70.0
Performance Trend: Improving
Grade: C
```

10. Conclusion

The Student Performance Trend Analysis program uses Python conditional statements and arithmetic operations to calculate average marks, identify performance trends, and assign grades. It helps summarize a student's results and understand their progress across three tests.